Benchmarking YOLO and DETR-Based Architectures for Object Detection on Small-Object-Biased Public Datasets and Analyzing the Impact of Training Strategies on Small-Object Detection Performance

Oren Elashri
Bachelor's Thesis in AI, 344.095, Summer Semester 2025/26
Silicon Austria Labs
oren.elashri@silicon-austria.com
July 16, 2026

Abstract

Object detection is a fundamental computer vision task, yet detecting small objects remains challenging due to limited visual information and sensitivity to dataset characteristics. This thesis **benchmarks two state-of-the-art detectors, YOLO11l and RT-DETR-l, on small-object-focused subsets of MS COCO 2017 and OpenImages V7**, comparing detection accuracy, inference latency, and robustness to data-centric training strategies. Three research questions are addressed. First, the COCO-pretrained models are evaluated on both datasets using $\mathbf{mAP}_{50:95}$, $\mathbf{AP}_{\mathbf{small}}$, $\mathbf{AP50}_{\mathbf{small}}$ and latency, establishing a baseline for cross-dataset comparison. Second, cross-dataset generalization is analyzed by fine-tuning both models on OpenImages V7 and evaluating them on both OpenImages V7 and COCO using $\mathbf{AP}_{\mathbf{small}}$, $\mathbf{AP}_{\mathbf{medium}}$, $\mathbf{AP}_{\mathbf{large}}$ and $\mathbf{mAP}_{50:95}$, capturing the effect of **dataset bias** in both transfer directions (COCO to OI and vise versa). Third, the **impact of Mosaic, MixUp, and Oversampling** on small-object detection is examined. Results show that **YOLO11l achieves higher in-distribution accuracy and substantially lower latency, while RT-DETR-l generalizes more robustly under cross-dataset transfer**, despite persistently low $\mathbf{AP}_{\mathbf{small}}$ for both models. Transferring the COCO-pretrained models to OpenImages causes severe $\mathbf{AP}_{\mathbf{small}}$ degradation, whereas fine-tuning on OpenImages and evaluating back on COCO slightly improves $\mathbf{AP}_{\mathbf{small}}$ at the cost of $\mathbf{mAP}_{50:95}$. The effectiveness of training strategies is **architecture-dependent**: Mosaic benefits YOLO11l in-domain, while MixUp is more effective for RT-DETR-l; oversampling raises in-domain $\mathbf{AP}_{\mathbf{small}}$ but fails to transfer consistently across datasets. These findings **underline the importance of jointly considering dataset properties and architecture-specific training strategies when optimizing small-object detection.**

1 Introduction

Object detection is a core task in computer vision, involving the simultaneous localization and classification of objects within an image. Over the past decade, deep learning has reshaped this field. Convolutional neural network (CNN) architectures first set strong baselines through two stage detectors such as Faster R CNN [1], which separate region proposal from classification, followed by one stage detectors like the YOLO family [2], which unify both steps for real time performance. More recently, transformer based architectures introduced global self attention as an alternative to convolution, enabling richer contextual reasoning but with increased computational cost [3, 4].

Despite these advances, detecting small objects - those occupying only a few pixels - remains a persistent challenge. Downsampling in deep networks reduces spatial resolution, leaving limited feature information for small scale instances [5]. Moreover, small objects are underrepresented in major benchmarks such as MS COCO 2017 [6], both in frequency and spatial eminence. As a result, models with strong overall mean Average Precision (mAP) may still perform poorly on small object subsets, with further degradation on various datasets with different scale distributions, such as OpenImages V7 [7].

This thesis addresses these limitations through a structured benchmarking study, "Benchmarking YOLO and DETR Based Architectures for Object Detection on Small Object Biased Public Datasets and Analyzing the Impact of Training Strategies on Small Object Detection Performance." Two representative modern architectures are evaluated: YOLO11l [8], a state of the art single stage convolutional detector with $\sim$ 25M parameters, and RT-DETR-l [4], a hybrid CNN transformer real time detector with $\sim$ 33M parameters. The evaluation spans MS COCO 2017[6] and OpenImages V7 [7], enabling assessment of both cross dataset generalization and raw detection performance.

1.1 Motivation

The motivation for this work arises from two converging observations. First, single-stage detectors from the YOLO family have become dominant in real-time object detection owing to their favorable accuracy-latency trade-off [2, 8]. Recent variants, including YOLO11l, incorporate enhanced feature aggregation and attention mechanisms that improve detection across object scales, building on advances such as feature pyramid networks (FPN) [5] to better represent objects at varying sizes. Second, transformer-based architectures such as DETR [3] and its real-time successor RT-DETR [4] introduce a fundamentally different paradigm: rather than relying on dense grid predictions anchored to fixed spatial locations, they predict a fixed set of objects using global self-attention, which in principle enables richer contextual reasoning.

However, it remains unclear whether the global reasoning capabilities of transformer-based detectors confer advantages specifically for small objects, which occupy only a small portion of the image and may benefit less from long-range context than from high-resolution spatial detail. Deformable DETR [9] addressed some of the efficiency and small-object limitations of the original DETR through deformable attention, and RT-DETR extends this with a hybrid CNN encoder. Nevertheless, a rigorous comparative evaluation of these architectures under small-object-dominated conditions, across datasets with different scale distributions, is still lacking in the literature.

Several benchmarking studies evaluate detection models on a single dataset, which limits the ability to distinguish between true model generalization and dataset-specific bias [6, 10]. This limitation is particularly critical, as dataset bias has been shown to substantially influence detection performance, especially for small objects [11].

The pronounced differences in small-object frequency and scale distribution between MS COCO 2017 and OpenImages V7 provide a suitable testbed for analyzing cross-dataset generalization. In addition to architectural variations, the role of training strategies aimed at mitigating class imbalance and the underrepresentation of small objects [12] - remains insufficiently explored across both convolutional and transformer-based detectors. Evaluating whether such strategies yield consistent performance improvements is of practical importance for applications in which small objects are dominant, including aerial imagery, medical imaging, and traffic monitoring.

This work builds on a preceding practical benchmarking exercise that evaluated multiple architectures under controlled conditions [13]. The present thesis extends that foundation into a structured academic analysis, focusing on training strategies, cross-dataset generalization, small-object detection performance, and the interaction between them.

1.2 Research Questions

This work addresses the following research questions:

1. Which model - YOLO11l (25M parameters)[8] or RT-DETR-l (32M parameters)[4] - achieves the best **trade-off between detection accuracy and inference latency** when evaluated on small-object-focused subsets of MS COCO 2017[6] and OpenImages V7[7]?

2. How does **dataset bias** - specifically the differences in small-object frequency and scale distribution between MS COCO 2017[6] and OpenImages V7[7] - affect small-object detection performance (mAP_{50-95}, AP_{small}, latency) and cross-dataset generalization of the two evaluated architectures?
3. How do augmentation and data-centric training strategies - specifically Mosaic [14], MixUp [15], and oversampling [12] - influence **small-object detection performance and cross-dataset generalization** of YOLO11l [8] and RT-DETR-l [4] when trained on OpenImages V7 [7] and evaluated across both OpenImages V7 and MS COCO 2017 [6]?

These questions are addressed through a controlled, reproducible benchmarking framework applied to both models under identical evaluation conditions, enabling direct and fair comparison across architectures and datasets.

1.3 Outline

The remainder of this thesis is structured as follows:

- **Chapter 2 - Material and Methods:** Describes the two datasets used (MS COCO 2017[6] and OpenImages V7[7]), including the adopted small-object definition and the derived 24 classes subset. It then introduces the two evaluated object detection architectures, covering related work on one-stage detectors (YOLO family)[2, 8], transformer-based detectors (DETR, RT-DETR)[3, 4], and multi-scale feature pyramid networks[5], followed by technical summaries of YOLO11l and RT-DETR-l. The chapter then presents the data augmentation strategies applied (oversampling, copy-paste, and the Ultralytics augmentation pipeline), and concludes with the evaluation metrics used to assess detection performance.
- **Chapter 3 - Implementation:** Details the reproducible benchmarking framework, including experimental setup, hardware, preprocessing pipeline, evaluation protocol, and key code components enabling transparent experimentation.
- **Chapter 4 - Results:** Presents results aligned with the research questions from Section 1.2. Metrics include $mAP_{50:95}$, AP_{small}, AP_{medium}, AP_{large}, and inference latency, reported for both architectures across both datasets and their small-object-focused subsets.
- **Chapter 5 - Alternative Methods:** Reviews state-of-the-art small-object detection approaches not implemented in this work. It discusses Slicing Aided Hyper Inference (SAHI)[16], which improves detection via image tiling, and Scale-Aware Relay / Scale-Adaptive Loss (SARL/SAL)[17], which enhance feature representation and loss behavior for tiny objects, particularly in aerial imagery.
- **Chapter 6 - Discussion:** Interprets findings relative to the research questions, examines cross-dataset performance differences, outlines study limitations, and suggests future work, including augmentation strategies and scaling to larger model families.

2 Material and Methods

This chapter describes the datasets, object detection models, data augmentation strategies, and evaluation metrics employed to benchmark small-object detection performance across COCO 2017 and OpenImages V7.

2.1 Datasets Description

2.1.1 Definition of Small Objects

The concept of a small object in this thesis follows the official definition introduced by the MS COCO benchmark dataset. According to the COCO evaluation protocol, objects are categorized based on their detection area into three scale groups: small, medium, and large. An object is considered **small** if its area is smaller than $32^2 = 1024$ pixels, **medium** if its area lies between 32^2 and 96^2 pixels, and **large** if its area exceeds $96^2 = 9216$ pixels [6].

This standardized definition was adopted to ensure consistency with previous literature and to enable direct utilization of established evaluation metrics such as AP_{small}. Using the COCO definition

avoids introducing dataset-specific thresholds that could bias comparisons between models and datasets. The COCO authors reported that approximately 41% of all annotated objects in their training set belong to the small-object category [6], highlighting the importance of scale-aware evaluation in object detection research and justifying the focus on small-object performance in this work.

2.1.2 MS COCO 2017

The Microsoft Common Objects in Context (MS COCO) 2017 dataset serves as the primary benchmark in this study. It contains complex everyday scenes with substantial variation in object scale, viewpoint, occlusion, and background clutter [6]. In this work, a **task-specific subset** is used, built by first identifying **24 target classes** selected for their highest proportion of small objects (area $< 32^2$ pixels), following the class-selection methodology developed in the author's prior practical work [13]. Because a single COCO image typically contains objects from several categories at once, selecting an image for its 24-class content does not remove its other annotations: each retained image keeps all of its original instance annotations, spanning the full set of 80 COCO categories, not just the 24 target classes used for selection. As a result, this image subset comprises approximately 122,000 training images with 896,782 annotated instances in total, drawn from all 80 categories.

Key characteristics of the COCO 2017 training set used in this work are summarized in Table 1 and Fig 1:

COCO 24 CLASSES - TRAINING SET - GLOBAL STATISTICS	
Metric	Value
Total training images	122,218
Number of object categories	80
Total annotated instances	896,782
Average objects per image	7.34
Average small objects/image	2.284
Images with $\geq$1 small object	53,214 (43.54%)
Small-object instance percentage	31.13%
Average small-object area	$\sim$397 px^2
Median small-object area	337 px^2
Mean image resolution	578 $\times$ 484 px

Table 1: COCO dataset-level statistics for small objects (area $< 32^2 = 1024$ px^2)

The relatively large proportion of small objects makes COCO particularly suitable for investigating small-object detection performance. Furthermore, the dataset provides the official COCO evaluation framework with dedicated metrics (AP_{small}, AP_{medium}, AP_{large}), which has become the standard for reporting object detection results in the literature [6].

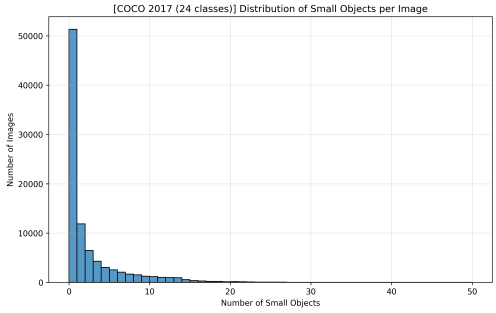
2.1.3 OpenImages V7

To evaluate cross-dataset generalization, the OpenImages V7 (OI V7) dataset was selected as a complementary benchmark. OpenImages is a large-scale dataset developed by Google that contains approximately 1.9 million images and more than 16 million bounding-box annotations distributed across 600 object categories [7]. The dataset provides much greater scale and diversity than COCO, with significantly different characteristics as shown in Table 2.

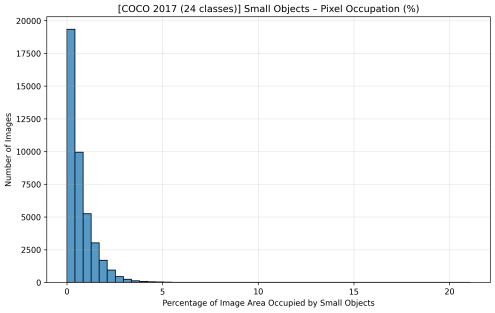
Compared with COCO, OpenImages exhibits substantially different characteristics: larger image resolutions, lower average object density per image, significantly smaller proportion of small objects, and different image acquisition sources and annotation procedures. These differences create a realistic domain-shift scenario for evaluating the robustness of object detection models.

Key statistics and differences between COCO and OpenImages V7 are summarized in Table 2:

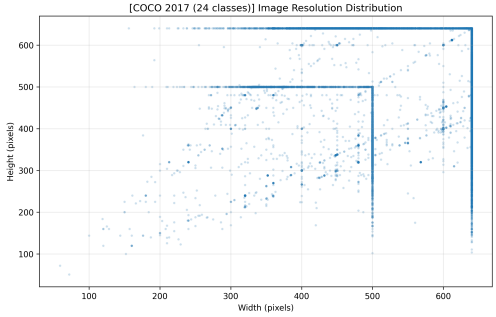
Preliminary experiments showed that models pretrained on COCO exhibit a substantial decrease in $\mathbf{AP_{small}}$ when evaluated on OpenImages, indicating strong sensitivity to dataset distribution shifts. This observation motivates the **cross-dataset** evaluation framework adopted in this study [13].



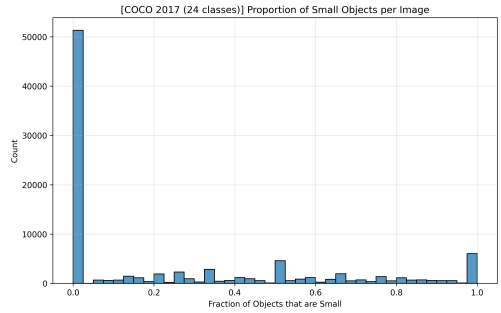
(a) Small objects per image.



(b) Pixel-area occupation of small objects.



(c) Image resolution distribution.



(d) Proportion of small objects relative to all instances per image.

Figure 1: Overview of small-object characteristics in the COCO 2017 training subset described in Table 1 (122,218 images, 80 categories, 896,782 annotated instances), including per-image counts, pixel-area occupation, image resolution distribution, and the proportion of small objects.

COCO 24 CLASSES TRAIN vs. OpenImages V7 - Dataset Characteristics		
Characteristic	COCO 24 CLASSES TRAIN	OpenImages V7
Number of images	122,218	$\sim$1.9 million
Object categories	80	600
Total annotations	896,782	$\sim$16 million
Small-object proportion	31.13%	13.39%

Table 2: Comparison of dataset-level characteristics between MS COCO 2017 and OpenImages V7.

2.1.4 OpenImages V7 - 24 Class Subset

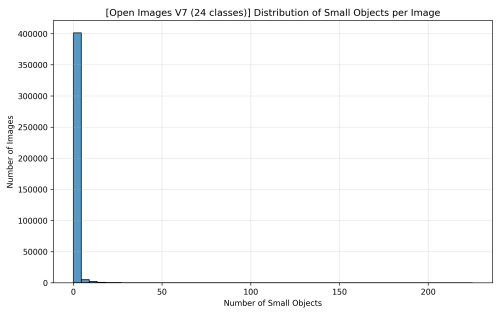
A direct comparison between COCO and OpenImages is complicated by their different label spaces (80 vs. 600 categories). Therefore, a custom subset consisting of 24 object categories shared between both datasets was constructed. This subset enables controlled, class-balanced comparison of model performance across the two datasets without confounding architectural effects with category composition differences. The selected 24 classes were derived from the most common and small-object-prevalent categories identified in COCO and satisfy the following criteria: (i) a high proportion of small instances (area $< 32^2$ px^2) among the category's annotations in COCO; (ii) the category is present in **both** COCO and OpenImages V7, ensuring the class is represented and usable across both datasets; (iii) the selected classes were informally checked to span multiple object domains (e.g., not exclusively vehicles or exclusively animals), avoiding a subset dominated by a single object type; and (iv) a directly corresponding category exists in OpenImages V7, enabling label-space alignment between the two datasets.

The resulting dataset subset contains the following statistics in Table 3 and Fig 2:

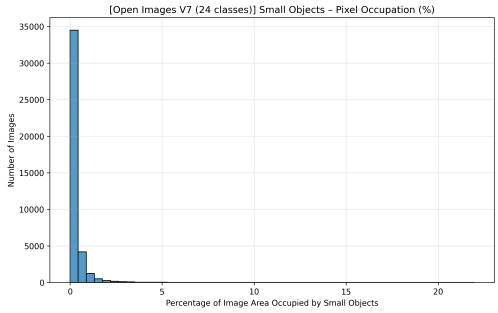
The 24 selected object categories are: *Person, Bicycle, Car, Motorcycle, Bus, Train, Truck, Boat, Bird, Cat, Dog, Horse, Sheep, Cow, Bottle, Chair, Potted Plant, Backpack, Handbag, Suitcase, Umbrella, Bench, Traffic Light, and Stop Sign.* By maintaining an identical class set across datasets, this benchmark isolates the influence of model architecture and data augmentation strategies from

OPENIMAGES 24 CLASSES SUBSET - TRAIN STATISTICS	
Metric	Value
Training images	410,260
Estimated total instances	$\sim$1.55 million
Average objects per image	3.78
Average small objects/image	0.419
Images with $\geq$1 small object	41,194 (10.04%)
Small-object instance percentage	11.03%
Mean image resolution	970 $\times$ 784 pixels

Table 3: OpenImages 24 classes subset statistics for small objects ($< 32^2$ pixels).



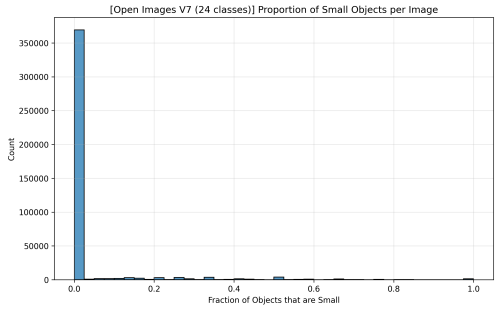
(a) Small objects per image.



(b) Pixel-area occupation of small objects.



(c) Image resolution distribution.



(d) Proportion of small objects relative to all instances per image.

Figure 2: Overview of small-object characteristics in the OpenImages V7 24-class subset **training set** described in Table 3 (410,260 images, $\sim$1.55 million annotated instances), including per-image counts, pixel-area occupation, image resolution distribution, and the proportion of small objects.

differences in category composition, allowing for a fair cross-dataset generalization assessment. As shown in Fig. 3, COCO and OpenImages exhibit markedly different distributions of small objects per image and small-object areas, underscoring the importance of dataset-specific characteristics in small-object detection.

2.2 Object Detection Models

This study compares two modern object detection paradigms: the YOLO family of one-stage detectors and Transformer-based DETR detectors. These represent fundamentally different architectural philosophies - dense anchor-free prediction versus sparse set prediction with global attention - making them ideal candidates for investigating the role of architecture in small-object detection.

2.2.1 YOLO11-Large (YOLO11l)

The You Only Look Once (YOLO) family represents one-stage object detectors that perform object localization and classification in a single forward pass. Unlike traditional two-stage detectors (e.g.,

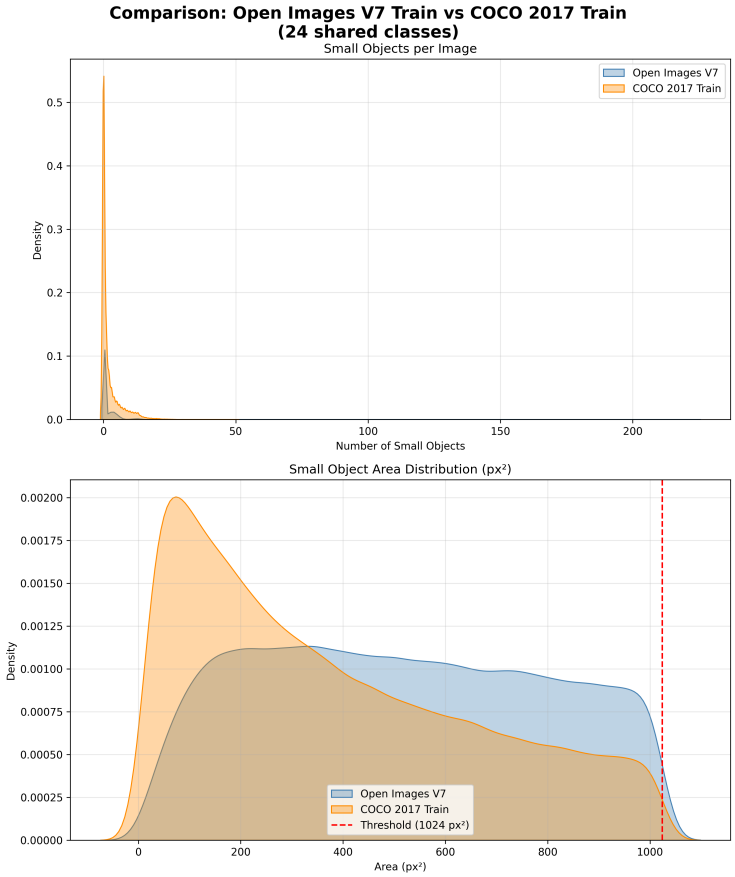


Figure 3: Comparison of small-object statistics for the COCO and OpenImages 24-class subsets. The **top panel** contrasts the number of small objects per image, while the **bottom panel** compares the distribution of small-object areas. Together, these visualizations illustrate the substantially higher small-object density and smaller average object scale in COCO relative to OpenImages.

Faster R-CNN) [2], YOLO directly predicts bounding boxes and class probabilities from image features, resulting in high inference speed and computational efficiency [2].

This study utilizes YOLO11-Large (YOLO11l), the large-variant implementation provided by Ultralytics [8], containing more than 25 million parameters. Architecturally, YOLO11l follows the standard three-stage YOLO design: a **backbone** for feature extraction, a **neck** (PANet-based) for multi-scale feature aggregation [5], and an **anchor-free detection head** that regresses bounding-box offsets directly alongside per-class probability vectors, rather than matching predictions to a predefined set of anchor boxes [13]. This design allows more flexible adaptation across object scales. YOLO11 additionally introduces attention-based C2PSA modules, first proposed in YOLOv10 and extended in YOLO11, which enhance spatial feature representation prior to detection [13, 8, 18, 19].

The main advantages of YOLO11l include: (i) low inference latency relative to comparable detectors, enabling near real-time deployment [8, 20]; (ii) efficient deployment on resource-constrained edge devices [8, 20]; (iii) a strong performance-to-speed trade-off compared to other modern detectors, as demonstrated in independent benchmarking studies [8]; (iv) multi-scale feature learning through feature pyramid networks and path aggregation [5, 8, 20]; and (v) anchor-free prediction, reducing the number of design hyperparameters [20, 8].

2.2.2 RT-DETR-Large (RT-DETR-l)

Detection Transformer (DETR) reformulates object detection as a direct set prediction problem using Transformer architectures [3]. Instead of generating numerous candidate regions followed by Non-Maximum Suppression (NMS) [21], DETR predicts a fixed set of objects and uses bipartite matching during training. The original DETR introduced several important innovations [3]: (i) end-

to-end differentiable object detection, (ii) global context modeling through self-attention mechanisms, (iii) elimination of handcrafted anchor generation, and (iv) removal of the NMS post-processing step. However, it suffered from slow convergence, high computational requirements, and limited performance on small objects [9, 4].

To address these limitations, this work utilizes RT-DETR-Large (RT-DETR-l), a real-time DETR variant with $\sim$33 million parameters, optimized for practical deployment while retaining the benefits of Transformer-based global reasoning [4]. Architecturally, RT-DETR-l processes images through a CNN backbone that extracts multi-scale feature maps, passed to a transformer encoder-decoder that performs global reasoning via self-attention, using a fixed-size set of learned object queries to directly predict bounding boxes and class labels [13]. RT-DETR-l refines this design through a hybrid CNN-transformer backbone with cross-scale feature fusion (CCFF), a multi-scale feature encoder, and a reduced number of object queries, collectively improving inference efficiency while preserving global context modeling [3, 4, 13].

Compared with YOLO11l, RT-DETR-l offers stronger global context modeling through multi-head self-attention, capturing long-range dependencies across the image, and a design free of handcrafted anchor scaling, aspect ratios, and NMS tuning. At the same time, Transformer-based detectors generally exhibit higher per-image computational cost and inference latency than purely convolutional approaches, and the theoretical benefit of global attention for small objects - which occupy minimal image area - remains an open question.

2.3 Data Augmentation Strategies

Several augmentation strategies were investigated in this thesis to assess their impact on small-object detection performance: (i) targeted small-object oversampling with Copy-Paste augmentation, and (ii) a comprehensive augmentation pipeline derived from modern YOLO training procedures. Both strategies aim to address two data-related challenges inherent in object detection datasets: **class imbalance**, where certain object categories are represented by substantially fewer annotated instances than others, and **underrepresentation of small objects**, where small-object instances (area $< 32^2$ px^2) contribute disproportionately little to the training loss relative to medium and large objects, causing the optimization to be dominated by larger objects despite small objects being a primary focus of this work.

2.3.1 Oversampling and Copy-Paste Augmentation

The first strategy is based on the Copy-Paste augmentation approach proposed by Kisantal et al. [12], augmented with targeted oversampling of images containing small objects. Copy-Paste augmentation **generates new training examples by extracting segmented objects from one image and pasting them into another image** at random locations. This synthetic augmentation increases object diversity and sample count while preserving realistic object appearance and scale relationships.

To further emphasize small-object learning, images containing at least one small object (area $< 32^2$ px^2) were **oversampled**: for an oversampling factor of $N\times$, each such image appears N times in the training set per epoch, consisting of the original unmodified image plus $(N-1)$ additional copy-paste-augmented variants, each generated with a different random placement of pasted small objects. This construction follows Kisantal et al. [12], who define an oversampling factor of two as retaining both the original image and one augmented copy of it. Images without small objects appear only once per epoch (no oversampling applied). Three oversampling factors were evaluated: $2\times$ (1 original + 1 augmented copy), $3\times$ (1 original + 2 augmented copies), and $4\times$ (1 original + 3 augmented copies).

The expected benefits of this strategy include:

- Increased frequency of small-object observations, compensating for class imbalance.
- Improved representation of underrepresented spatial scales in the feature learning process.
- Better model robustness to object occlusion and complex backgrounds through synthetic occlusion scenarios.

Potential drawbacks and risks include:

- Distribution shift toward small objects, which may reduce performance on large and medium objects.
- Increased risk of overfitting due to repeated samples and synthetic artifacts.
- Artificial object placement artifacts (e.g., unnatural overlaps) potentially introducing training noise.

2.3.2 Ultralytics Augmentation Pipeline

The second strategy follows the comprehensive augmentation pipeline commonly used in modern YOLO training frameworks [8, 22, 23, 24] This pipeline applies few complementary augmentation techniques sequentially, each addressing different aspects of model robustness and generalization.

Mosaic Augmentation. Mosaic augmentation combines four independent training images into a single composite image [14, 24]. This technique exposes the model to objects at multiple scales and spatial positions within a single forward pass, increases contextual diversity, and effectively enlarges the receptive field. Mosaic has been shown to significantly improve detection performance, particularly for small objects that benefit from increased contextual information.

MixUp Augmentation. MixUp generates synthetic samples by linearly combining two images and their corresponding class labels [14, 15, 24]. This regularization strategy improves model generalization, reduces overfitting to specific image patterns, and has been shown to enhance robustness on out-of-distribution datasets. Mathematically, MixUp blends a randomly drawn pair of training examples as [15]:

$$\tilde{x} = \lambda x + (1 - \lambda)x', \qquad \tilde{y} = \lambda y + (1 - \lambda)y',$$

where x and x' are the raw input images of the two sampled examples, y and y' are their corresponding one-hot encoded class labels, $\tilde{x}$ and $\tilde{y}$ are the resulting mixed image and label, and $\lambda \in [0, 1]$ is a mixing coefficient controlling the interpolation ratio between the two examples.

2.4 Evaluation Metrics

Model performance was assessed using the official COCO evaluation protocol as implemented in the pycocotools library [25], which provides standardized metrics enabling fair and reproducible comparison of detection models across datasets and with results reported in the literature [6].

The primary aggregate metric is mean Average Precision (mAP), computed as the mean over ten uniformly spaced Intersection over Union (IoU) thresholds ranging from 0.50 to 0.95 in increments of 0.05, formally denoted mAP@[0.50:0.95] or mAP_{50-95}. As established by Lin et al. [6], the headline COCO metric (`stats[0]`) corresponds to :

$$mAP_{[50-95]} = \frac{1}{10} \sum_{t \in \mathcal{T}} AP^{(t)}$$

where $\mathcal{T} = \{0.50, 0.55, \ldots, 0.95\}$ and $AP^{(t)}$ denotes the Average Precision computed at a single IoU threshold t. At each threshold t, a predicted bounding box is counted as a true positive only if its IoU with the matched ground-truth box exceeds t; otherwise it is counted as a false positive. $AP^{(t)}$ is then computed from the area under the resulting precision-recall curve, aggregated over all images and categories.

Scale-stratified performance is reported using AP_{small} (area $< 32^2$ px^2), AP_{medium} ($32^2 - 96^2$ px^2), and AP_{large} ($> 96^2$ px^2), each averaged over ten IoU thresholds [6]. Since aggregate mAP is dominated by larger instances, such stratification is **essential** for evaluating small-object detection.

AP_{small} serves as the primary metric. It is the most challenging due to two factors: small localization errors cause significant IoU degradation, particularly at strict thresholds ($t \geq 0.75$), and input downsampling (e.g., 640×640 px) can reduce objects to only a few pixels, leading to severe information loss [26].

For OpenImages, annotations were converted to COCO JSON with validated bounding boxes and category mappings, ensuring **cross-dataset** comparability [26].

3 Implementation

3.1 Experimental Setup

Experiments were conducted on a shared high-performance computing cluster (NVIDIA A100, 80GB VRAM) using SLURM, with one GPU per run. Training used a batch size of 8 for 35 epochs. Latency was measured with 80 warm-up and 150 timed batches. All models employed **AMP** during training and FP16 inference.

3.2 System Architecture

1. **Dataset & Models Downloader -** *1_download_datasets.py* - Automates the download and preparation of the MS COCO2017, OpenImages V7 datasets, and the model weights needed.
2. **Dataset Analysis -** *2_dataset_analysis.py* - Analyzes COCO and OpenImages V7 annotations to compute small-object statistics per class, ranks classes by small-object percentage, and generates the corresponding visualizations.
3. **Preprocess OI 24 Classes Dataset -** *3_prepare_oi_yolo_24classes.py* - Load from Open Images only images with at least one of the 24 classes that were common between Open Images and COCO datasets - for train and val, including labels and .yaml file.
4. **OI 24 Classes Dataset Analysis -** *4_analyze_oi_24classes.py* - Computes small-object statistics (count, ratio, pixel occupation) for the OpenImages 24-class subset and compares them against the corresponding COCO 2017 statistics, to characterize dataset-level differences relevant to small-object detection.
5. **Training Pipeline** - Unified training script for all models, Controlled hyperparameters across experiments.
   (a) AUGMENTATION TRAIN MODELS - *5_train_experiments.py* - Create 3 different training strategies: Baseline (no augmentation), Mosaic, and Mixup. On YOLO11l and RT-DETR-l, run in batches for 35 epochs.
   (b) OVERSAMPLE TRAIN MODELS - *6_train_oversampling.py* - Create 2 oversampling different training strategies: 2x, 3x. On YOLO11l and RT-DETR-l, run in batches for 35 epochs.
6. **Evaluation Pipeline -** *7_compare_experiments.py* - Evaluate fine-tuned YOLO11l and RT-DETR-l models using official COCOeval metrics and realistic GPU inference benchmarking.

3.3 Training Pipeline Architecture

Two complementary experimental strategies were investigated to assess the impact of data augmentation and small-object oversampling on detection performance.

Augmentation Strategies. Three training configurations were evaluated by varying **only the augmentation setting under test**, while keeping all remaining hyperparameters - including all other augmentation techniques - fixed across experiments: ***Baseline*** (no augmentation), ***Mosaic*** (probability $= 1.0$), and ***MixUp*** (probability $= 0.1$). No other augmentations were applied in any of the three configurations. Both YOLO11l and RT-DETR-l were trained for **35 epochs** with a batch size of 8, early-stopping patience of 20 epochs, a **dataset fraction of 0.9** (i.e., 90% of the training images randomly sampled per epoch, rather than the full training set), a reduced initial learning rate with stronger final decay, gradual learning-rate warm-up, a fixed random seed for reproducibility, and ℓ_2 weight-decay regularization.

Oversampling Strategies. To further address the spatial underrepresentation of small objects, two targeted oversampling configurations were evaluated: $\mathbf{2}\times$ and $\mathbf{3}\times$ repetition of images containing at least one small-object instance (area $< 32^2$ px^2). All remaining training hyperparameters were held **identical** to those described above, with an early-stopping patience of 20 for both architectures.

Listing 1: Controlled multi-strategy training pipeline. Each architecture is trained across the three augmentation configurations introduced above - ***Baseline*** (no augmentation), ***Mosaic***, and ***MixUp*** -

under identical hyperparameters, ensuring **comparability**. Fixed seeds, consistent learning schedules, and standardized data loading isolate the effect of augmentation policies on detection performance while maintaining reproducible experimental conditions [22, 23].

```
for model_pt, model_name in MODELS:
    print(f"\n Training {model_name.upper()} with aug strategies...\n")
    for strategy_name, aug_params in STRATEGIES.items():
        run_name = f"{model_name}_{strategy_name}"
        print(f"   → Running: {run_name}")
        model = YOLO(model_pt)

        model.train(
            data=os.path.join(YOLO_DATA_ROOT, "data.yaml"),
            epochs=EPOCHS, imgsz=IMG_SIZE, batch=8, device=DEVICE,
            project=PROJECT_DIR, name=run_name, exist_ok=True,
            patience=20, amp=True, cache="disk", workers=4, seed=42,
            resume=False, fraction=0.9,

            # Learning Rate
            lr0=0.001, lrf=0.01, warmup_epochs=5.0, warmup_bias_lr=0.1,
            weight_decay=0.0005, label_smoothing=0.0,

            # Save & Validate
            val=True, save=True, save_period=5,

            # Strategy-specific augmentations
            **aug_params,

            # Common augmentations
            **no_augs,)
```

3.4 Evaluation Pipeline Architecture

The benchmarking framework centers on a unified evaluation pipeline designed to measure both detection accuracy and inference efficiency under controlled and reproducible experimental conditions. Particular emphasis is placed on small-object detection performance, assessed using the official COCO evaluation protocol.

The evaluation pipeline consists of four main modules:

1. **Data Loader.** This component belongs to the **evaluation pipeline**, run after training to compute the final benchmarking metrics reported in this thesis (mAP, AP_{small}, latency, FPS). Images from each dataset's standard held-out split - COCO's *val2017* and OpenImages' *val* subset - are loaded using two dataset-specific classes, *`CocoSmallObjectDataset`* and *`OpenImagesSmallObjectDataset`*, restricted to the 24 matched categories. Each image is decoded, converted to RGB (if needed), and resized to 640×640 pixels via *`torchvision.transforms`*. Images are grouped into batches of 16 using a multi-worker *`DataLoader`* with pinned memory to maximize GPU transfer throughput. The same evaluation-split images are used for both accuracy and latency measurement, ensuring pre-processing overhead is included in latency estimates. Evaluation is conducted independently on the MS COCO 24-class and OpenImages V7 24-class **evaluation** splits. COCO ground truth is taken from the official annotation file; OpenImages ground truth is generated in COCO format from the YOLO-style label files.

2. **Model Module.** All fine-tuned model weights are loaded sequentially via the Ultralytics *YOLO* interface, which supports both YOLO11 and RT-DETR architectures. Each model is transferred to the target GPU device prior to evaluation. The total number of learnable parameters is recorded in millions by summing all parameter tensors in the model. Models are evaluated in inference mode *(torch.inference_mode)* throughout, with FP16 half-precision enabled to reduce GPU memory consumption and improve computational throughput without affecting reported metric values.
3. **Inference Latency Measurement.** Inference throughput is benchmarked using a two-phase protocol: an 80-batch warmup to stabilise GPU clock frequencies, followed by timing over 150 subsequent complete batches, with CUDA synchronisation *(torch.cuda.synchronize)* applied before and after each forward pass. Per-image latency is computed as the median elapsed time divided by batch size, and FPS is derived as its reciprocal.
4. **Accuracy Evaluation.** Following inference, model predictions are exported in COCO-compatible JSON format. Detection accuracy is subsequently computed using the official *COCOeval* toolkit from *pycocotools*, which evaluates predictions against the ground-truth annotation file for each dataset. The following metrics are extracted directly from the COCOeval statistics array: $\text{mAP}_{[0.50:0.95]}$ *(stats[0])*, AP_{small} *(stats[3])*, AP_{medium} *(stats[4])*, and AP_{large} *(stats[5])*. Results across all models and datasets are consolidated into a structured DataFrame, ranked in descending order of mAP@[0.50:0.95]. A latency-versus-AP_{small} scatter plot is generated per dataset, showing all evaluated models simultaneously. All results are persisted to CSV and JSON formats for subsequent analysis.

Listing 2: Unified category remapping across heterogeneous label schemas. Since COCO and OpenImages V7 use different category ID systems, both are remapped onto a single, consistent 24-class taxonomy *(MODEL_CLASS_MAP)*. This mapping is generated once from the matched-class name set and reused throughout the pipeline, ensuring that predictions from either dataset are directly comparable under a common label space [6, 7].

```
MATCHED_CLASSES = {
    'apple', 'backpack', 'baseball glove', 'bird',
    'boat', 'book', 'bottle', 'car', 'clock', 'handbag',
    'kite', 'knife', 'mouse', 'orange', 'parking meter',
    'person', 'sheep', 'snowboard', 'spoon', 'tie',
    'toothbrush', 'traffic light', 'vase', 'wine glass'}

# Unified 24-class taxonomy shared by both datasets
MODEL_CLASS_MAP = {name: idx for idx, name in
enumerate(sorted(MATCHED_CLASSES))}

# COCO's native category IDs → unified taxonomy
COCO_MATCHED_IDS = {
    'person': 0, 'car': 2, 'boat': 8, 'traffic light': 9,
    'parking meter': 12, 'bird': 14, 'sheep': 18, 'backpack': 24,
    # ... remaining classes
    }

COCO_TO_MODEL = {
    coco_id: MODEL_CLASS_MAP[name]
    for name, coco_id in COCO_MATCHED_IDS.items()}
```

Listing 3: Small-object-focused accuracy evaluation via class and image-filtered COCOeval. Rather than evaluating on the full COCO category set, predictions and ground truth are restricted to the 24 matched classes and the images containing them. This isolates the metric most relevant to the

thesis objective, AP_{small}, from the standard COCOeval statistics array [27, 25], avoiding dilution of the score by unrelated categories.

```
coco_gt = COCO(COCO_ANN)
coco_dt = coco_gt.loadRes(pred_path)
coco_eval = COCOeval(coco_gt, coco_dt, "bbox")

# Restrict evaluation to the 24 matched classes and their images
matched_cat_ids = [dataset.class_name_to_id[cls] for cls in
MATCHED_CLASSES]
matched_img_ids = sorted(set(
    img_id for cat_id in matched_cat_ids
    for img_id in coco_gt.getImgIds(catIds=[cat_id])
))

coco_eval.params.catIds = matched_cat_ids
coco_eval.params.imgIds = matched_img_ids
coco_eval.evaluate()
coco_eval.accumulate()
coco_eval.summarize()

metrics = {
    "mAP50_95": float(coco_eval.stats[0]),
    "AP_small": float(coco_eval.stats[3]),
    "AP_medium": float(coco_eval.stats[4]),
    "AP_large": float(coco_eval.stats[5]),}
```

4 Results

This chapter presents the experimental results addressing the three research questions formulated in Section 1.2. All experiments employed the OpenImages V7 24-class subset (Section 2.1.4) as the primary training domain, with cross-dataset evaluation on the MS COCO 24-class subset (Section 2.1.2) to assess generalization robustness.

4.1 Augmentation Strategy Training

The following experiments report performance on the standard OpenImages V7 validation split, as computed automatically by the Ultralytics training framework at the end of each training run [22, 23]. This validation-based comparison is used specifically to **select the best-performing augmentation strategy per architecture** (Mosaic for YOLO11l, MixUp for RT-DETR-l); no separate in-domain test set was held out at this stage, consistent with common practice for strategy selection during model development. The selected strategies are subsequently evaluated on the held-out MS COCO 24-class subset (Section 2.1.2) to assess cross-dataset generalization, as presented in Sections 4.2 and 4.3.

4.1.1 YOLO11l on OpenImages V7

This section evaluates five distinct training strategies to isolate the impact of individual augmentation techniques on YOLO11l's optimization dynamics and validation performance. The strategies compared are: (1) **Baseline** with no augmentation, (2) **MixUp**, (3) **Mosaic**, (4) **2× class oversampling**, and (5) **3× class oversampling**. Copy-Paste augmentation [28] was excluded, as the OpenImages V7 24-class subset contains only detection annotations. Since this method requires segmentation masks for object extraction, it remains **inapplicable** under the given experimental configuration.

Model	Precision	Recall	$\mathbf{mAP}_{50}$	$\mathbf{mAP}_{50:95}$
Baseline	0.5253	0.5181	0.5170	0.4148
MixUp	0.5475	0.5331	0.5249	0.4270
Mosaic	**0.5614**	**0.5457**	**0.5629**	**0.4617**
Oversample 2×	0.5562	0.5076	0.5270	0.4300
Oversample 3×	0.5562	0.5126	0.5159	0.4153

Table 4: Validation performance across augmentation strategies for the YOLO11l model.

Mosaic augmentation demonstrated superior performance across all metrics. It achieved the highest precision (56.1%), recall (54.6%), mAP_{50} (56.3%), and $mAP_{50:95}$ (46.2%). Critically, Mosaic also exhibited the fastest convergence and the lowest validation losses across bounding-box regression, classification, and focal loss components, indicating both superior optimization efficiency and reduced generalization error. MixUp and 2× oversampling yielded moderate improvements (+3-4% $mAP_{50:95}$), suggesting that moderate regularization and class balancing enhance robustness. However, 3× oversampling degraded performance almost below baseline, with substantially prolonged convergence, demonstrating that excessive sample repetition reduces effective sample diversity and promotes memorization. These results indicate that **data augmentation benefits YOLO11l training on OpenImages V7**, but the **choice of strategy significantly impacts** both **optimization efficiency and final generalization**. Mosaic augmentation provides the most favorable trade-off between convergence speed and detection accuracy (Tables 4), making it the preferred augmentation strategy for subsequent cross-dataset generalization experiments.

4.1.2 RT-DETR-l on OpenImages V7

This section extends the augmentation study to the RT-DETR-l architecture in order to evaluate whether the trends observed for YOLO11l generalize to a transformer-based detector. **RT-DETR differs substantially from conventional CNN-based detectors** through its transformer encoder-decoder design and query-based prediction mechanism. All experiments were conducted under identical training conditions to ensure a fair comparison.

Model	Precision	Recall	$\mathbf{mAP}_{50}$	$\mathbf{mAP}_{50:95}$
Baseline	0.5137	0.5226	0.5248	0.4221
MixUp	0.4992	**0.5625**	**0.5340**	**0.4284**
Mosaic	0.5036	0.5185	0.5190	0.4077
Oversample 2×	0.5143	0.5379	0.5293	0.4188
Oversample 3×	**0.5620**	0.4997	0.5062	0.4057

Table 5: Validation performance across augmentation strategies for the RT-DETR-l model.

The results show that the effectiveness of augmentation is architecture-dependent. **Unlike YOLO11l, RT-DETR-l did not benefit from Mosaic augmentation,** as shown in Table 5, which produced low overall performance with an $mAP_{50:95}$ of 0.4077, indicating that the strong geometric transformations negatively affected the transformer's learning process. In contrast, **MixUp achieved the best overall results**, obtaining the highest recall (0.5625), mAP_{50} (0.5340), and $mAP_{50:95}$ (0.4284), outperforming the Baseline (0.5226 recall, 0.5248 mAP_{50}, and 0.4221 $mAP_{50:95}$). **Oversampling produced mixed results.** Oversample 2× slightly improved precision (0.5143), recall (0.5379) and mAP_{50} (0.5293) while maintaining performance close to the baseline. **Increasing the oversampling factor** to 3× resulted in the **highest precision (0.5620), but reduced recall (0.4997) and mAP performance**, suggesting that **excessive oversampling made the model more conservative.** Overall, MixUp provided the most balanced improvement for RT-DETR-l, demonstrating that augmentation strategies should be selected according to the detector architecture rather than applied universally.

4.2 Baseline Cross-Dataset Performance (No Augmentation)

This section evaluates the cross-dataset generalization of YOLO11l and RT-DETR-l when trained without augmentation strategies. Models were trained on two datasets (i) MS COCO 2017 (24 classes subset) and (ii) OpenImages V7 (24 classes subset), and evaluated on both to measure dataset shift sensitivity and identify which data distributions present challenges for each architecture. This experiment isolates dataset bias (RQ2) from the effects of augmentation-based training strategies.

In-Distribution Performance (COCO → COCO)

Table 6 reports the performance of both models evaluated on the MS COCO 2017 (24 classes) validation set under identical inference conditions (640×640 resolution, FP16 precision). YOLO11l achieves a higher overall $\text{mAP}_{50:95}$ of 0.5335 compared to RT-DETR-l's 0.5207, alongside a marginally superior AP_{small} of 0.3299 versus 0.3216. These results indicate that, within the COCO distribution, **YOLO11l maintains a consistent performance advantage across both overall and small-object detection metrics**. YOLO11l achieves this while using fewer parameters (25.33M vs. 32.86M) and substantially lower inference latency (3.43 ms vs. 7.13 ms), suggesting a more computationally efficient architecture for in-distribution tasks.

Model	Params (M)	Latency (ms)	$\mathbf{AP}_{\text{small}}$	$\mathbf{mAP}_{50:95}$	$\mathbf{AP50}_{\text{small}}$
YOLO11l	25.33	3.43	0.329850	0.5335	0.2869
RT-DETR-l	32.86	7.13	0.321591	0.5207	0.2765

Table 6: COCO-pretrained YOLO11l and RT-DETR-l evaluated on the MS COCO (24 classes) evaluation set (640×640, FP16 inference). This table corresponds to the COCO→COCO evaluation setting.

Cross-Dataset Generalization (COCO → OI)

Table 7 reports zero-shot transfer results on OpenImages V7. **The transfer causes a severe collapse in strict small-object localization**: AP_{small} drops from 0.3299 (YOLO11l) and 0.3216 (RT-DETR-l) in-distribution (Table 6) to just 0.0624 and 0.0655 respectively under transfer - a relative reduction of roughly 81% and 80%. This confirms that models pretrained exclusively on COCO fail to generalize their fine-grained small-object localization to a structurally different dataset such as OpenImages.

Model	Params (M)	Latency (ms)	$\mathbf{AP}_{\text{small}}$	$\mathbf{mAP}_{50:95}$	$\mathbf{AP50}_{\text{small}}$
YOLO11l	25.33	3.33	0.062388	0.3707	0.3907
RT-DETR-l	32.86	7.41	0.065549	0.4578	0.4537

Table 7: COCO-pretrained YOLO11l and RT-DETR-l evaluated on the OpenImages V7 (24 classes) evaluation set without fine-tuning (COCO→OI). This table highlights cross-dataset generalization performance under identical inference conditions.

Quantifying Performance Degradation Under Dataset Shift

Table 8 summarizes the absolute and relative degradation in AP_{small} when transferring from COCO to OpenImages. **Both architectures exhibit severe performance collapse in small-object detection under domain shift**. YOLO11l suffers an absolute AP_{small} drop of 0.2675. RT-DETR-l degrades by 0.2560. While neither model generalizes reliably to OpenImages without fine-tuning, these results highlight that **small-object detection is disproportionately vulnerable to dataset shift** relative to overall detection performance. The latency profiles remain broadly stable across datasets (-2.9% for YOLO11l, $+3.9\%$ for RT-DETR-l), confirming that the degradation is attributable to feature-level distributional mismatch rather than computational factors.

Model	$\Delta\text{AP}_{\text{small}}$	Latency Δ %
YOLO11l	0.2675	-2.9
RT-DETR-l	0.2560	$+3.9$

Table 8: Performance degradation when transferring from COCO to OpenImages without fine-tuning.

Taken together, these baseline results establish two key findings for the subsequent experimental analysis: (i) YOLO11l is the stronger in-distribution detector, and (ii) RT-DETR-l exhibits greater cross-dataset resilience, particularly under the challenging conditions of zero-shot transfer to a heterogeneous, web-crawled dataset such as OpenImages V7.

4.3 Impact of Augmentation on Cross-Dataset Generalization

This section evaluates whether augmentation strategies that influence in-domain performance also improve cross-dataset robustness (RQ3). Models trained on the OpenImages V7 24 classes subset

were evaluated on both the OpenImages validation set (OI→OI) and the MS COCO 2017 24 classes validation set (OI→COCO), with the performance gap between these settings serving as a proxy for generalization capability. Where architectural differences in augmentation sensitivity are observed, these findings also bear on RQ1.

In-Domain Performance (OI → OI)

Table 9 reports in-domain results. For YOLO11l, **oversampling strategies yield the highest** $\mathrm{AP}_{\mathrm{small}}$**,** with Oversample 3× achieving 0.1148 - the strongest result across all YOLO11l variants. However, this gain comes at the cost of $\mathrm{mAP}_{50:95}$, which declines from 0.4704 (Baseline) to 0.4140, suggesting that **oversampling skews the model toward small objects at the expense of overall detection quality**. Similarly, **RT-DETR-l**, the Oversample 3× configuration achieves the highest $\mathrm{AP}_{\mathrm{small}}$ (0.1114), followed by the **baseline**. Other augmentation strategies perform worse, indicating that augmentation provides limited benefit for RT-DETR-l even under in-domain conditions.

Model	Params (M)	$\mathbf{AP}_{\mathrm{small}}$	$\mathbf{AP}_{\mathrm{medium}}$	$\mathbf{AP}_{\mathrm{large}}$	$\mathbf{mAP}_{50:95}$
YOLO11l-Baseline	25.33	0.074766	0.2155	**0.5431**	**0.4704**
YOLO11l-MixUp	25.33	0.090298	**0.2159**	0.5172	0.4573
YOLO11l-Mosaic	25.33	0.072231	0.2128	0.5062	0.4472
YOLO11l-Oversample 2×	25.33	0.110126	0.2091	0.4831	0.4176
YOLO11l-Oversample 3×	25.33	**0.114830**	0.1977	0.4827	0.4140
RT-DETR-l-Baseline	32.86	0.105158	0.2096	**0.5503**	**0.4732**
RT-DETR-l-MixUp	32.86	0.102272	0.2106	0.5230	0.4473
RT-DETR-l-Mosaic	32.86	0.096039	0.1814	0.5267	0.4513
RT-DETR-l-Oversample 2×	32.86	0.091336	0.2005	0.5072	0.4352
RT-DETR-l-Oversample 3×	32.86	**0.111378**	**0.2192**	0.5261	0.4476

Table 9: OpenImages-trained YOLO11l and RT-DETR-l evaluated on OpenImages evaluation (OI→OI).

Cross-Dataset Transfer (OI → COCO)

Table 10 reveals a notable changing of the in-domain augmentation gains. **YOLO11l-Mosaic achieve the highest** $\mathrm{AP}_{\mathrm{small}}$ of 0.095966 under cross-dataset transfer, slightly outperforming YOLO11l-Baseline (0.0928), while Oversample 3× - the strongest in-domain variant - degrades to 0.0872, the worst among all YOLO11l configurations. Contrarily, **RT-DETR-l-Baseline retains the highest** $\mathrm{AP}_{\mathrm{small}}$ of 0.1379 on COCO, with MixUp, Oversample 3× and Mosaic reducing performance to 0.1337, 0.1321 and 0.1195, respectively. Table 11 shows that $\mathrm{AP}_{\mathrm{small}}$ increases for both architectures Baseline under domain transfer (+0.0181 for YOLO11l and +0.0327 for RT-DETR-l), while $\mathrm{AP}_{\mathrm{medium}}$ also improves substantially. In contrast, $\mathrm{AP}_{\mathrm{large}}$ and $\mathrm{mAP}_{50:95}$ decrease notably, indicating that **domain transfer prioritizes smaller objects** at the expense of large-object and overall detection performance.

Across both architectures, **Oversample 3× yields the strongest in-domain** $AP_{\mathbf{small}}$ (Table 9), but this advantage collapses **for YOLO11l under cross-dataset** evaluation, where **Baseline and Mosaic perform best** instead (Table 10). **RT-DETR-l's Baseline remains the strongest transfer configuration overall.** This suggests **augmentation gains are architecture and domain-dependent.** These findings address RQ2: augmentation improves in-domain small-object detection selectively, but is not a reliable mechanism for cross-dataset robustness.

4.4 Architectural Comparison: YOLO11l vs. RT-DETR-l (RQ1)

Within-distribution, YOLO11l outperforms RT-DETR-l on both $\mathrm{AP}_{\mathrm{small}}$ (0.3299 vs. 0.3216) and $\mathrm{mAP}_{50:95}$ (0.5335 vs. 0.5207), while operating at less than half the inference latency (3.43 ms vs. 7.13 ms) with $\sim$ 23% fewer parameters. Under cross-dataset transfer, however, this advantage reverses: **RT-DETR-l generalizes substantially better to unseen distributions**, achieving a higher $\mathrm{mAP}_{50:95}$ on OpenImages (0.4578 vs. 0.3707). YOLO11l is therefore preferred for latency-critical in-distribution deployments, while RT-DETR-l is the stronger choice when cross-domain robustness is the priority.

Model	Params (M)	$\mathbf{AP}_{\text{small}}$	$\mathbf{AP}_{\text{medium}}$	$\mathbf{AP}_{\text{large}}$	$\mathbf{mAP}_{50:95}$
YOLO11l-Baseline	25.33	0.092818	0.3253	**0.4522**	0.2044
YOLO11l-MixUp	25.33	0.091889	0.3199	0.4396	0.2014
YOLO11l-Mosaic	25.33	**0.095966**	**0.3400**	0.4386	**0.2105**
YOLO11l-Oversample 2×	25.33	0.092655	0.3021	0.3892	0.1835
YOLO11l-Oversample 3×	25.33	0.087240	0.2978	0.3822	0.1789
RT-DETR-l-Baseline	32.86	**0.137906**	0.3818	**0.5073**	**0.2440**
RT-DETR-l-MixUp	32.86	0.133719	0.3788	0.4939	0.2370
RT-DETR-l-Mosaic	32.86	0.119503	0.3623	0.4895	0.2304
RT-DETR-l-Oversample 2×	32.86	0.131537	0.3836	0.5053	0.2437
RT-DETR-l-Oversample 3×	32.86	0.132149	**0.3837**	0.5031	0.2427

Table 10: OpenImages-trained YOLO11l and RT-DETR-l evaluated on MS COCO evaluation (OI→COCO).

Model	ΔAP_{small}	$\Delta AP_{\text{medium}}$	ΔAP_{large}	$\Delta\text{mAP}_{50:95}$
YOLO11l-Baseline	0.0181	0.1098	-0.0909	-0.2660
RT-DETR-l-Baseline	0.0327	0.1722	-0.0430	-0.2292

Table 11: Change in AP_{small}, AP_{medium}, AP_{large}, and $\text{mAP}_{50:95}$ when OpenImages-trained models are evaluated on COCO (OI→COCO).

5 Alternative Methods

Several advanced techniques for improving small object detection were reviewed during the literature study but were not included in the experimental evaluation of this thesis. Although these methods have demonstrated promising improvements in detection accuracy, their integration would have introduced additional variables that could compromise the controlled comparison of the investigated models. Moreover, implementing and evaluating these techniques would have required substantial additional computational time and development effort, exceeding the practical time constraints of the current experimental framework.

5.1 Slicing Aided Hyper Inference (SAHI)

One of the most influential approaches for improving small-object detection is Slicing Aided Hyper Inference (SAHI), proposed by Akyon et al. [16]. SAHI addresses the challenge that small objects occupy only a limited number of pixels in high-resolution images, making them difficult for conventional object detectors to identify, as illustrated in Fig. 4.

The SAHI framework consists of two complementary components:

- **Slicing Aided Fine-Tuning (SF)** - Training images are divided into smaller patches before training. The patches are then resized, effectively increasing the relative size of small objects and allowing the detector to learn more discriminative features.
- **Slicing Aided Hyper Inference (SAHI)** - During inference, the original image is partitioned into overlapping patches. Each patch is processed independently by the detector, after which the predictions are merged back into the original image using Non-Maximum Suppression (NMS) or similar post-processing techniques. Optionally, an additional full-image inference pass can be performed to improve the detection of large objects [16].

The authors evaluated SAHI using the anchor-free detectors **FCOS** [29], **VFNet** [30], and **TOOD** [31] on the **VisDrone** [32, 33] and **xView** [34] aerial-image datasets, which contain large number of small objects. Experimental results showed that SAHI alone increased Average Precision (AP) by **6.8%, 5.1%, and 5.3%** for FCOS, VFNet, and TOOD, respectively. When combined with Slicing Aided Fine-Tuning (SF), the cumulative AP improvements increased to **12.7%, 13.4%, and 14.5%** compared to full-image inference baselines. Furthermore, the use of overlapping patches (approximately 25% overlap) was found to be particularly beneficial for improving performance on small and medium-sized objects [16].

Despite these advantages, SAHI introduces several practical limitations. Since each image must be processed multiple times, inference latency increases significantly. Additional overlap between patches further increases computational requirements and may introduce duplicate detections that require careful post-processing. Moreover, large-object detection performance can worsen when full-image inference is omitted [16].

For the purposes of this thesis, SAHI was not incorporated into the main experimental pipeline. The primary reason is that the method substantially increases inference time, making fair latency comparisons between different detector architectures more difficult. Since one objective of this work is to compare models under consistent inference conditions, introducing a slicing-based inference strategy would have altered the computational characteristics of the evaluated detectors and potentially biased the comparison.

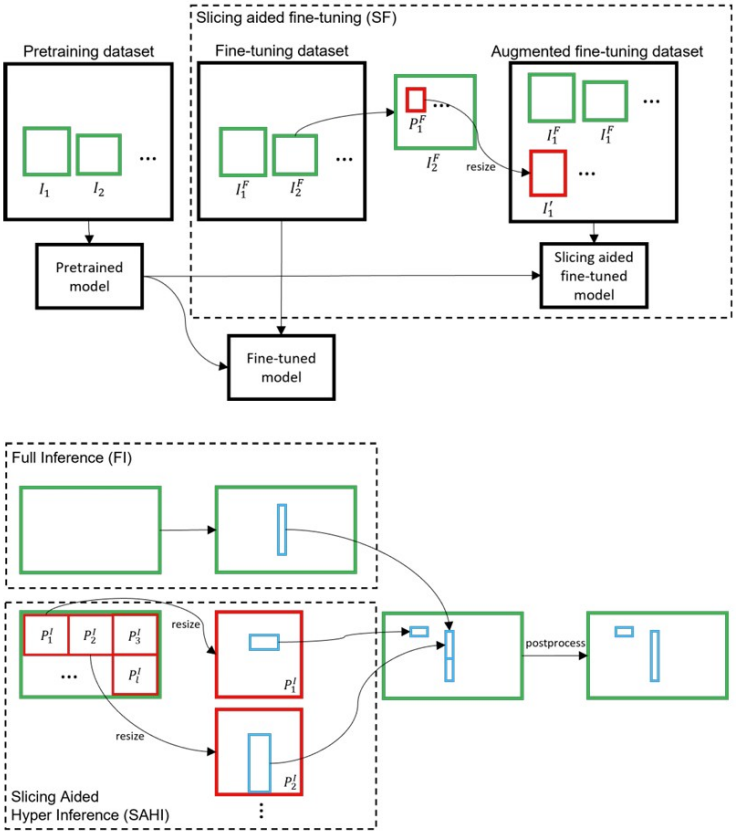


Figure 4: Overview of slicing-aided fine-tuning and slicing-aided inference. Images are split into smaller patches, resized, and processed to improve detection of small objects. Predictions are mapped back to the original image after non-maximum suppression [16].

5.2 Scale-Aware Methods (SARL and SAL)

Another category of methods focuses on explicitly improving the representation and learning of tiny objects through architectural modifications and specialized loss functions. A recent example is the work of Li et al. [17], who proposed **Scale-Aware Relay Layers (SARL) and a Scale-Adaptive Loss (SAL) for tiny-object detection in aerial imagery.**

The motivation behind these methods is that **tiny objects often lose important feature information during propagation through deep neural networks.** Furthermore, conventional IoU-based regression losses **tend to penalize localization errors disproportionately for small objects**, making optimization more challenging [17].

The proposed approach introduces:

- **Scale-Aware Relay Layers (SARL):** feature-enhancement modules that employ cross-scale spatial-channel attention mechanisms to improve information exchange between feature levels and preserve small-object characteristics.
- **Scale-Adaptive Loss (SAL):** a modified IoU-based loss function that dynamically assigns lower weights to larger objects, thereby focusing the optimization process more strongly on tiny targets [17].

Experimental evaluation on the **AI-TOD** [35], **DOTA-v2.0** [36], and **VisDrone2019** [32, 33] datasets demonstrated an improvement of approximately **5.5% AP** when the proposed components were integrated into both anchor-based and anchor-free detection frameworks [17].

Although these results are promising, SARL and SAL require modifications to the internal architecture and training objectives of the detector. Such changes conflict with the methodological goal of this thesis, which is to compare object detection models under a controlled framework while minimizing alterations to their original implementations. Consequently, scale-aware architectural and loss-function modifications were considered outside the scope of this work.

6 Discussion

6.1 Conclusions - based on the RQ's

RQ1. Within-distribution (COCO→COCO), YOLO11l achieved both higher detection accuracy (AP_{small}, $\text{mAP}_{50:95}$) and substantially lower inference latency than RT-DETR-l, indicating that no accuracy-latency trade-off is required when the evaluation distribution matches the training distribution. Under cross-dataset transfer to OpenImages V7, however, this advantage reversed: RT-DETR-l attained higher AP_{small} and $\text{mAP}_{50:95}$ despite its higher latency. **The optimal accuracy-latency trade-off is therefore context-dependent: YOLO11l is preferable for latency-critical, in-distribution deployment, while RT-DETR-l offers superior robustness at greater computational cost when the target distribution is unknown or heterogeneous.**

RQ2. Dataset bias strongly influences small-object detection, with transfer direction determining degradation patterns. Transfer from COCO to OpenImages V7 causes a severe drop in AP_{small} for both models. In contrast, transfer from OpenImages V7 to COCO slightly improves AP_{small}, but reduces AP_{large} and $\text{mAP}_{50:95}$. This asymmetry reflects that transfer from OpenImages V7 to COCO yields slightly improved small-object performance, whereas the opposite direction leads to stronger degradation. However, no definitive conclusion about dataset properties can be drawn. **RT-DETR-l remains more robust to dataset shift than YOLO11l.**

RQ3. The effectiveness of data-centric training strategies depends **on both architecture and evaluation setting**. In-domain (validation), Mosaic provides the most balanced improvement for YOLO11l, while MixUp is most effective for RT-DETR-l, with Mosaic degrading transformer performance. On test evaluation, oversampling maximizes AP_{small} for both models but reduces overall accuracy. These gains do not transfer consistently: under cross-dataset evaluation, oversampling degrades YOLO11l to the weakest AP_{small}, while Baseline and Mosaic perform better; for RT-DETR-l, no augmentation surpasses Baseline. **No strategy is consistently optimal across architectures or settings.**

6.2 Limitations

Several limitations should be considered when interpreting the results presented in this thesis.

1. Although **Copy-Paste augmentation** was initially included in the experimental design, it was **omitted from the final analysis**. Early experiments showed no measurable improvements over the configuration. Furthermore, Copy-Paste was originally developed for instance segmentation tasks, where object masks enable object extraction and placement. In this work, only bounding-box annotations were available, limiting its effectiveness for object detection.

2. The experiments were conducted using a **common set of training hyperparameters** rather than performing an extensive hyperparameter optimization for each model and augmentation strategy.

Consequently, **the reported results should be interpreted as a comparison of the relative effects of different training strategies** under identical conditions rather than the maximum achievable performance of each individual method.

3. Statistical validation. Each configuration was trained with a single run and fixed random seed due to high computational cost. Reported differences therefore reflect observations rather than statistically validated improvements. Since deep learning training involves randomness, results can vary between runs. Future work should therefore repeat experiments with multiple random seeds and report confidence intervals or statistical significance tests to verify that the observed differences are reliable and not due to chance.

4. Only two public datasets were considered. Although both **MS COCO 2017 and OpenImages V7 contain a large number of object categories**, they **differ considerably in image resolution, annotation style, object scale distribution, and scene complexity**. These differences influence detector performance and make direct comparisons challenging. **The experiments suggest** that **OpenImages V7 presents a more difficult detection task** than MS COCO 2017, although further investigation would be required to isolate the individual causes of this difference.

5. Only one representative model size from each detector family was evaluated. While YOLO11l and RT-DETR-l provide a meaningful comparison between convolution-based and transformer-based object detectors, the observed trends may not fully generalize to other model sizes or more recent detector architectures.

6. Finally, several observations regarding training behaviour are based on empirical evidence rather than dedicated architectural analysis. For example, RT-DETR-l consistently showed degraded performance when trained with Mosaic augmentation. This behaviour is likely related to the transformer's reliance on global contextual information and object-query matching, whereas Mosaic introduces artificial spatial discontinuities by combining multiple scenes into a single image. However, confirming this hypothesis would require additional feature-level and attention representations analysis.

6.3 Outlook

Several directions can extend this work. A natural next step is automated hyperparameter optimization for each detector and augmentation strategy, including combined strategies, to assess whether observed trends persist under individually optimized settings.

Future studies should evaluate more recent architectures, larger model variants, and additional datasets with diverse object-scale distributions. Incorporating domain-specific datasets, such as aerial or traffic imagery, would further improve insight into small-object detection.

Another promising direction is the integration of alternative approaches such as SAHI and SARL/SAL. Finally, analyzing transformer attention maps and intermediate feature representations may explain why augmentation strategies, particularly Mosaic, affect RT-DETR-l differently from convolutional detectors.

References

[1] Shaoqing Ren et al. Faster r-cnn: Towards real-time object detection with region proposal networks. In *NeurIPS*, 2015.

[2] Joseph Redmon et al. You only look once: Unified, real-time object detection. In *CVPR*, 2016.

[3] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *ECCV*, 2020.

[4] Yian Zhao et al. Detrs beat yolos on real-time object detection. *arXiv preprint arXiv:2304.08069*, 2023.

[5] Tsung-Yi Lin et al. Feature pyramid networks for object detection. In *CVPR*, 2017.

[6] Tsung-Yi Lin et al. Microsoft coco: Common objects in context. In *ECCV*, 2014.

[7] Alina Kuznetsova et al. The open images dataset v4. *International Journal of Computer Vision*, 128:1956–1981, 2020.

[8] Glenn Jocher and Jing Qiu. Ultralytics yolo11, 2024. Accessed: 2026.

[9] Xizhou Zhu et al. Deformable detr: Deformable transformers for end-to-end object detection. In *ICLR*, 2021.

[10] Pierre-Luc Asselin, Vincent Coulombe, William Guimont-Martin, and William Larrivée-Hardy. Replication study and benchmarking of real-time object detection models. *arXiv preprint arXiv:2405.06911*, 2025. Version 2, 04 Sep 2025. License: CC BY 4.0.

[11] Antonio Torralba and Alexei A. Efros. Unbiased look at dataset bias. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011.

[12] Mate Kisantal, Zbigniew Wojna, Jakub Murawski, Jacek Naruniec, and Kyunghyun Cho. Augmentation for small object detection. In *BMVC*, 2019.

[13] Oren Elashri. Benchmarking yolo and detr-based architectures for object detection on small-object-biased public datasets. Technical report, Johannes Kepler Universität Linz / Silicon Austria Labs, 2026. Practical Work course report.

[14] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. Yolov4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.

[15] Hongyi Zhang et al. mixup: Beyond empirical risk minimization. In *ICLR*, 2018.

[16] Fatih Cagatay Akyon et al. Slicing aided hyper inference and fine-tuning for small object detection. *arXiv preprint arXiv:2202.06934*, 2022.

[17] Zhen Liu, Yifan Wang, Hao Chen, and Ming Li. Scale-aware relay and scale-adaptive loss for tiny object detection. *arXiv preprint arXiv:2511.09891*, 2025. Proposes SARL (Scale-Aware Relay Layer) and SAL (Scale-Adaptive Loss).

[18] Ao Wang, Hui Chen, Lihao Liu, et al. Yolov10: Real-time end-to-end object detection. *arXiv preprint arXiv:2405.14458*, 2024.

[19] Leo Thomas Ramos and Angel D. Sappa. A decade of you only look once (yolo) for object detection. *arXiv preprint arXiv:2504.18586*, 2025.

[20] Rehana Khanam and Muhammad Hussain. Yolov11: An overview of the key architectural enhancements, 2024.

[21] Jan Hosang, Rodrigo Benenson, and Bernt Schiele. Learning non-maximum suppression. In *CVPR*, 2017.

[22] Ultralytics. Ultralytics yolo documentation: Training modes and augmentation settings. `https://docs.ultralytics.com/modes/train#augmentation-settings-and-hyperparameters`, 2024. Accessed: 2026.

[23] Ultralytics. Ultralytics yolo documentation: Random erasing augmentation. `https://docs.ultralytics.com/guides/yolo-data-augmentation`, 2024. Accessed: 2026.

[24] Ultralytics. Yolov5 documentation: Model architecture and data augmentation. `https://docs.ultralytics.com/yolov5/tutorials/architecture_description/`, 2023. Accessed: 2026.

[25] twsl and contributors. pytorchcocotools: Coco api for pytorch. `https://github.com/twsl/pytorchcocotools`, 2020. GitHub repository. Accessed: 2026.

[26] Shweta Singh, Aayan Yadav, Jitesh Jain, Humphrey Shi, Justin Johnson, and Karan Desai. Benchmarking object detectors with coco: A new path forward. *arXiv preprint arXiv:2403.18819*, 2024.

[27] PyTorch Contributors. PyTorch Documentation: Benchmarking. `https://pytorch.org/docs/stable/benchmark_utils.html`, 2025. Accessed: 2026.

[28] Golnaz Ghiasi et al. Simple copy-paste is a strong data augmentation method for instance segmentation. In *CVPR*, 2021.

[29] Zhi Tian, Chunhua Shen, Hao Chen, and Tong He. Fcos: Fully convolutional one-stage object detection. *arXiv preprint arXiv:1904.01355*, 2019.

[30] Haoyang Zhang, Ying Wang, Feras Dayoub, and Niko Sünderhauf. Varifocalnet: An iou-aware dense object detector. *arXiv preprint arXiv:2008.13367*, 2020.

[31] Chengjian Feng, Yujie Zhong, Yu Gao, Matthew R. Scott, and Weilin Huang. Tood: Task-aligned one-stage object detection. *arXiv preprint arXiv:2108.07755*, 2021.

[32] Pengfei Zhu, Longyin Wen, Dawei Du, Xiao Bian, Qinghua Hu, and Haibin Ling. Vision meets drones: Past, present and future. *arXiv preprint arXiv:2001.06303*, 2020.

[33] Dawei Du, Longyin Wen, Pengfei Zhu, Heng Fan, et al. Visdrone-cc2020: The vision meets drone crowd counting challenge results. *arXiv preprint arXiv:2107.08766*, 2021.

[34] Darius Lam, Richard Kuzma, Kevin McGee, Samuel Dooley, Michael Laielli, Matthew Klaric, Yaroslav Bulatov, and Brendan McCord. xview: Objects in context in overhead imagery. *arXiv preprint arXiv:1802.07856*, 2018.

[35] Chun-Han Tsui, Tsung-Yi Lin, Ross Girshick, Piotr Dollár, and C. Lawrence Zitnick. Ai-tod: Aerial tiny object detection. In *ICCV*, 2021.

[36] Gui-Song Xia, Xiang Bai, Jian Ding, Zhen Zhu, Serge Belongie, Jiebo Luo, Mihai Datcu, Marcello Pelillo, and Liangpei Zhang. Dota: A large-scale dataset for object detection in aerial images. In *CVPR*, 2018.